

Proof under Total Correctness 1: Avoid Runtime Errors in Loop-free Programs

So far, all the proof of triples (formal proofs, full proof outlines, minimal and partial proof outlines) and all the axioms and inference rules we learned are for partial correctness (although most of them work for both correctness). Each proof system we have seen is a proof system for partial correctness, which is the set of logical formulas determined by the sets of axioms and rules of inference for partial correctness.

Now let's see how to come up with a **proof system for total correctness** which can prove a triple $\{p\} S \{q\}$ provable under total correctness: $\vdash_{tot} \{p\} S \{q\}$: we need to avoid possible runtime errors and divergence. Here, let's look at how to avoid runtime errors in loop-free programs first.

- Previously, we have learned how to get wp from wlp : we include domain predicates for expressions $D(e)$, predicates $D(p)$, and statements $D(S)$ that guarantee the evaluation of e, p and S won't cause a runtime error.
- We use a similar idea to get $\vdash_{tot} \{p'\} S \{q'\}$ from $\vdash \{p\} S \{q\}$ by including domain predicates:
 - In general, for a loop-free S :
 - If we get $\vdash \{wlp(S, q)\} S \{q\}$, then for total correctness we have $\vdash_{tot} \{wp(S, q \wedge D(q))\} S \{q \wedge D(q)\}$.
 - If we get $\vdash \{p\} S \{sp(p, S)\}$, then for total correctness we have $\vdash_{tot} \{p \wedge D(p) \wedge D(S)\} S \{sp(p \wedge D(p) \wedge D(S), S)\}$.
- For a predicate p , we define the **safe version** of p as $\downarrow p \equiv p \wedge D(p)$.
 - With this definition we can rewrite $wp(S, q) \Leftrightarrow D(S) \wedge \downarrow wlp(S, q)$
- If a predicate $p \Leftrightarrow \downarrow p$, then we say p is **safe**. x > \
- In a triple $\{p\} S \{q\}$, if p and q are both safe, then $\{p\} S \{q\}$ is **safe**. In a formal proof under total correctness, all the triple judgments should be **provable under total correctness and also safe**, so we cannot prove any unsafe triple. In this case, we need to use the safe version of the precondition and/or the postcondition while creating the proof.

(Inference Rules for Loop-free Statement under Total Correctness)

- We have the following non-loop axioms / inference rules for total correctness. Note that the antecedent triples and consequent triples in each rule are valid under total correctness.
 - Skip Axiom** (total correctness version):

$$\{\downarrow p\} \text{ skip } \{\downarrow p\} \quad \text{skip}$$
 - Backward Assignment Axiom** (total correctness version):

$$\{D(e) \wedge \downarrow ((\downarrow q) [e / v])\} v := e \{\downarrow q\} \quad \text{backward assignment}$$
 - Forward Assignment Axiom** (total correctness version):

$$\{D(e) \wedge \downarrow p\} v := e \{(D(e) \wedge \downarrow p)[v_0 / v] \wedge v = e[v_0 / v]\} \quad \text{forward assignment}$$
 - Sequence rule, strengthen precondition rule, weaken postcondition rule, conjunction rule, disjunction rule, conditional rule 1, nondeterministic conditional rule is still the same. The only difference is that now the antecedent triples and the consequent triples are safe and provable under total correctness.

For example, the conjunction rule:

1. $\{p_1\} S \{q_1\}$
2. $\{p_2\} S \{q_2\}$
3. $\{p_1 \wedge p_2\} S \{q_1 \wedge q_2\}$ conjunction 1,2

This rule still holds in the proof for total correctness, but now $\vdash_{tot} \{p_1\} S \{q_1\}$, $\vdash_{tot} \{p_2\} S \{q_2\}$, and p_1, q_1, p_2, q_2 are all safe, thus the consequent $\vdash_{tot} \{p_1 \wedge p_2\} S \{q_1 \wedge q_2\}$, and $p_1 \wedge p_2, q_1 \wedge q_2$ are safe.

There are two rules of inference that need to be updated for total correctness:

- **Conditional Rule 2** (total correctness version):
 1. $\{p_1\} S_1 \{q_1\}$
 2. $\{p_2\} S_2 \{q_2\}$
 3. $\{D(B) \wedge (B \rightarrow p_1) \wedge (\neg B \rightarrow p_2)\} \text{ if } B \text{ then } S_1 \text{ else } S_2 \text{ fi } \{q_1 \vee q_2\}$ if – else 1,2

Conditional rule 2 contains a fresh expression B in the antecedent.

- **Nondeterministic Conditional Rule** (total correctness version):
 1. $\{p \wedge B_1\} S_1 \{q_1\}$
 2. $\{p \wedge B_2\} S_2 \{q_2\}$
 3. $\{p \wedge (B_1 \vee B_2)\} \text{ if } B_1 \rightarrow S_1 \sqcap B_2 \rightarrow S_2 \text{ fi } \{q_1 \vee q_2\}$ if – fi 1,2

At least one of the guards needs to be true or else we will have a runtime error.

1. Create a full proof outline for $\{b[x] < y\} y := y + 1; z := y \{q\}$ under total correctness. Find a possible q .
 - First, we add domain predicate in the precondition, then we can use forward assignment to find a provable q .
 - We can create the following proof outline:

$$\{b[x] < y \wedge 0 \leq x < \text{size}(b)\}$$

$$y := y + 1; \{b[x] < y_0 \wedge 0 \leq x < \text{size}(b) \wedge y = y_0 + 1\}$$

$$z := y \{b[x] < y_0 \wedge 0 \leq x < \text{size}(b) \wedge y = y_0 + 1 \wedge z = y\}$$
2. Under total correctness, create a full proof outline for provable triple $\{p\} \text{ if } x \geq 0 \text{ then } x := \text{sqrt}(x) \text{ else } x := y/b[k] \text{ fi } \{0 \leq x < y\}$. Find a possible p .
 - The postcondition is already safe.
 - We can calculate the wp as p first. But in general, the calculation of wp can be time consuming, and we might not be able to prove the triple with wp as the precondition using the inference rules we have. Here, we use the conditional rule to create proof and come a provable precondition naturally.
 - We can create the following proof outline:

$$\{p\}$$

$$\text{ if } x \geq 0 \text{ then}$$

$$\{0 \leq \text{sqrt}(x) < y \wedge x \geq 0\} x := \text{sqrt}(x) \{0 \leq x < y\}$$

$$\text{ else}$$

$$\{0 \leq y/b[k] < y \wedge 0 \leq k < \text{size}(b) \wedge b[k] \neq 0\} x := y/b[k] \{0 \leq x < y\}$$

$$\text{ fi } \{0 \leq x < y\}$$

Where $p \equiv (x \geq 0 \rightarrow 0 \leq \text{sqrt}(x) < y \wedge x \geq 0) \wedge (x < 0 \rightarrow 0 \leq y/b[k] < y \wedge 0 \leq k < \text{size}(b) \wedge b[k] \neq 0)$
3. Under total correctness, create a full proof outline for provable triple $\{T\} \text{ if } x \geq 0 \text{ then } x := \text{sqrt}(x) \text{ else } x := y/b[k] \text{ fi } \{q\}$. Find a possible q .

- We find q naturally following the proof outline.
- T is a safe precondition, but the program has the probability to create a runtime error so we cannot take all states into consideration. To create proof under total correctness, we need to strengthen the precondition to $T \wedge D(IF) \equiv T \wedge D(x \geq 0) \wedge (x \geq 0 \rightarrow D(x := \text{sqrt}(x))) \wedge (x < 0 \rightarrow D(x := y/b[k]))$

$$\begin{aligned} &\Leftrightarrow (x \geq 0 \rightarrow x \geq 0) \wedge (x < 0 \rightarrow 0 \leq k < \text{size}(b) \wedge b[k] \neq 0) \\ &\Leftrightarrow x < 0 \rightarrow 0 \leq k < \text{size}(b) \wedge b[k] \neq 0 \\ &\Leftrightarrow x \geq 0 \vee 0 \leq k < \text{size}(b) \wedge b[k] \neq 0 \end{aligned}$$



- We can use Conditional Rule 1 to create the following proof outline:
 $\{x \geq 0 \vee 0 \leq k < \text{size}(b) \wedge b[k] \neq 0\}$
if $x \geq 0$ **then**
 $\{(x \geq 0 \vee 0 \leq k < \text{size}(b) \wedge b[k] \neq 0) \wedge x \geq 0\} \{x \geq 0\} x := \text{sqrt}(x) \{x_0 \geq 0 \wedge x = \text{sqrt}(x_0)\}$
else
 $\{(x < 0 \rightarrow 0 \leq k < \text{size}(b) \wedge b[k] \neq 0) \wedge x < 0\}$
 $\{0 \leq k < \text{size}(b) \wedge b[k] \neq 0\} x := y/b[k] \{0 \leq k < \text{size}(b) \wedge b[k] \neq 0 \wedge x = y/b[k]\}$
fi
 $\{x_0 \geq 0 \wedge x = \text{sqrt}(x_0) \vee 0 \leq k < \text{size}(b) \wedge b[k] \neq 0 \wedge x = y/b[k]\}$

Examples 1, 2 and 3 have other solutions, here I only provide one of the possible proofs.

Proof under Total Correctness 2: Proof of Convergence

- Remember that we only create proofs for provable triples. If a statement contains a loop that can be proved to be convergent, then this loop must iterate for a finite number of iterations.
 - For example, let $S \equiv k := 0; \text{while } k < 5 \text{ do } k := k + 1 \text{ od}$, then while loop iterates 5 iterations. When k gets increased to 2, including the current iteration, the loop still has 3 iterations.
- In general, we cannot always find the exact number of (remaining) iterations a loop has, but if we can upper bound the number of (remaining) iterations of a loop by a finite natural number and the number decreases after each iteration, then we can prove the loop converges.
 - For example, let $W \equiv \text{while } k < n \text{ do } k := k + 1 \text{ od}$, we don't have an exact number of iterations; but we can use $n - k$ to express the number of (remaining) iterations before W terminates, and $n - k$ is decreasing after each iteration.
- A **bound expression** or **bound function** t for a loop is a finite *natural number* expression that, at each loop test, gives a **strictly decreasing** upper bound on the number of iterations remaining before termination. A bound expression can use program variables (variables in statement) and logical variables (variables in precondition or postcondition).
- We'll attach the bound expression t to a loop using the syntax **{bd t }**, thus in a proof outline for total correctness, a loop usually looks like: **{inv p }{bd t } while B do S od**.

(Properties of Bound Expressions)

What properties do we need for t so that it can be bound for loop $W \equiv \{\text{inv } p\} \{\text{bd } t\} \text{ while } B \text{ do } S \text{ od}$? Here we list two requirements.

- $p \Rightarrow t \geq 0$.
 - Loop invariant p is always true after each iteration of a while loop, it needs to logically imply that there are non-negative iterations left to do.

- $\vdash_{tot} \{p \wedge B \wedge t = t_0\} S \{p \wedge t < t_0\}$
 - t_0 is the value of t before the execution of one iteration; after the execution of this iteration, the value of t needs to be strictly decreased to guarantee convergence.